

ブロック整列とループ回転 — 命令を減らせずに速くする

今日のゴール

ループが毎回実行している「条件へ戻るジャンプ」を消す。命令の個数は変わらないのに実行回数が減る、という最適化を体験する。

1 この回の位置づけ

- 最適化発展シリーズの第 7 回。前提は O1(測定基盤) と O2(フローグラフ)
- 編集するのは `layout.py` だけ。 `mycc.py` は書き換えない
- この回は静的命令数では効果が見えない。O1 で作った動的の物差しが必要になる

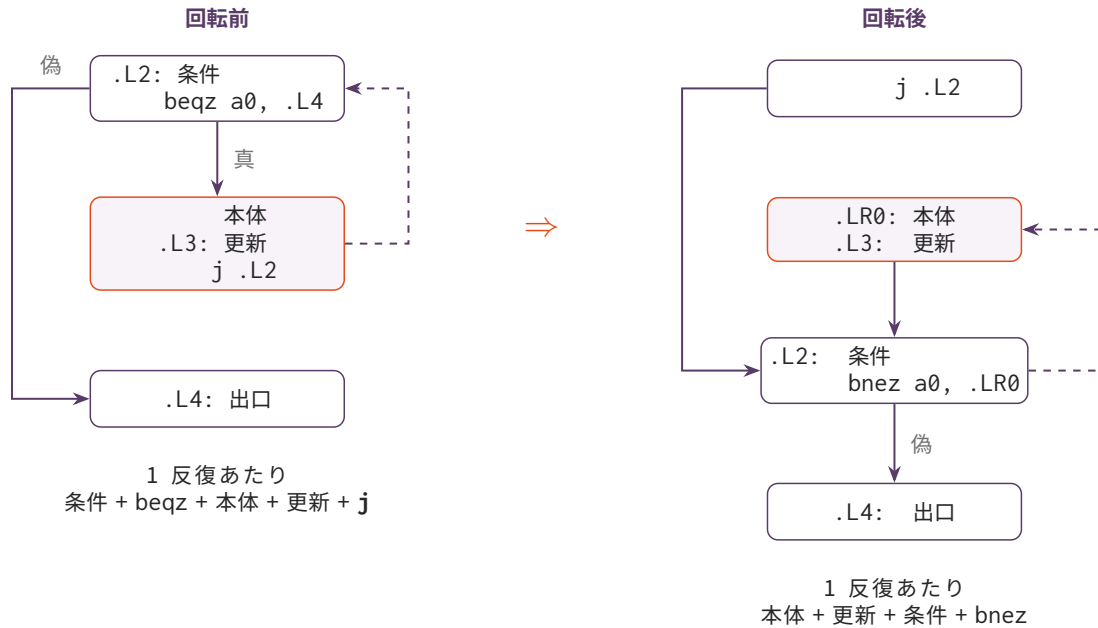
2 いまのループは何をしているか

コマ 6 で作った `while` の生成は、こういう形をしている。

```
.L2:                # ループの入口（条件）
    <条件の計算>
    beqz a0, .L4      # 偽なら出口へ
    <本体>
.L3:                # continue の飛び先
    <更新>
    j .L2             # 条件へ戻る
.L4:
```

素直な形である。ソースコードの `while (条件) { 本体 }` がそのまま見える。

しかし 1 回まわるたびに `j .L2` を実行している。条件判定がループの先頭にあるので、毎回そこへ戻らなければならない。



ループ1回につき **j** が1つ減る。入口の **j .L2** は最初の1回しか実行されない。
命令の個数は変わらない(1つ増えて1つ減る)。減るのは**実行される回数**のほうである — だから静的命令数では効果が見えず、動的命令数でしか測れない

図1 ループ回転: 1反復あたり **j** が1つ減る

3 ループ回転

条件をループの末尾へ動かすと、この **j** が要らなくなる。

```

    j .L2                # 最初の条件判定へ (1回だけ実行される)
.LR0:
    <本体>
.L3:
    <更新>
.L2:
    <条件の計算>
    bnez a0, .LR0        # 真なら本体へ戻る
.L4:
  
```

本体を実行したあと、そのまま下へ落ちて条件を計算し、真なら本体へ戻る。戻るための **j** が、条件分岐そのものに吸収された。

入口に **j .L2** が1つ増えるが、これはループに入るときの1回だけである。100回まわるループなら、99回分の **j** が消える。

重要

命令の個数は変わらない。`j` が 1 つ増えて、`j` が 1 つ減る。静的命令数は同じである。減るのは実行される回数のほうで、それは動的命令数でしか測れない。

4 条件の反転

条件を末尾へ動かすと、分岐の向きが逆になる。

前	後	意味
<code>beqz</code>	<code>bnez</code>	0 なら飛ぶ ↔ 0 でなければ飛ぶ
<code>beq</code>	<code>bne</code>	等しい ↔ 等しくない
<code>blt</code>	<code>bge</code>	小さい ↔ 以上
<code>bltu</code>	<code>bgeu</code>	(符号なし)

回転前は「条件が偽なら出口へ」だったものが、回転後は「条件が真なら本体へ」になるので、ちょうど反転する。

5 ループをどうやって見つけるか

ソースコードの `while` を探すのではない。**02** で作ったフローグラフの後方辺を探す。

```
blocks, edges = cfg.build_cfg(lines)
for tail, head in cfg.back_edges(blocks, edges):
    ... # head がループの入口
```

こうしておく、`while` でも `for` でも同じ仕組みで扱える。「ループ」をソースの構文ではなくグラフの性質として捉えることが要点である。

6 回してよい形かを確かめる

パターンが一致しても、機械的に書き換えると壊れることがある (03 と同じ教訓)。
`rotate_one` は次を確かめてから回す。

確認すること	なぜ
入口ラベルの直後が「条件 + 条件分岐」	途中に別のラベルがあると、そこへ飛んでくる経路がある
本体の最後が <code>j <入口></code>	戻り方が違う形は扱わない
その直後が出口ラベル	出口が本体の直後に無いと、並べ替えで到達できなくなる

いまのコンパイラが出す `while / for` はすべてこの形になる。条件を満たさないものは回さずにそのまま残す (安全側)。

`break` と `continue` は回転後もそのまま動く。`break` の飛び先 (出口ラベル) は動かさず、`continue` の飛び先 (更新のラベル) は本体と一緒に移動するからである。

7 ついでに: 次の行へのジャンプ

素の出力には、こういう並びがある。

```
j .L1
.L1:
ld s0, 16(sp)
```

`return` 文が関数の最後にあると出る形で、すぐ次の行へ飛んでいる。このジャンプは消してよい。こちらは静的にも動的にも 1 命令減る。

8 実装

Step	実装対象	内容
1	<code>invert_branch</code>	条件分岐の反転 (表を引くだけ)
2	<code>rotate_one</code> / <code>rotate_loops</code>	回転。形の確認がいちばん大事
3	<code>remove_jump_to_next</code>	次の行へのジャンプを消す
4	<code>run</code>	回転してから削除 (順番が逆だと消し損ねる)

ループの入口を探す `loop_headers` は、O2 の `cfg.py` を使って書いてある (完成済み)。

```
python3 ../optcc.py --passes isel,layout ../01_measure/bench/loop_sum.c
python3 ../optcc.py --passes isel          ../01_measure/bench/loop_sum.c
```

回転なし

9 テスト

```
python3 check.py      # 反転表と回転の単体テスト (未実装は SKIP)
python3 golden.py     # fixed15 + ベンチマーク + 静的/動的の比較
```

`golden.py` は動的命令数で合否を判定する。静的では効果が見えないためである。

10 測ってみると

教員の実装ではこうなる (`isel` をかけた状態を基準にしている)。

ベンチマーク	静的: 前	後	差	動的: 前	後	削減
bubble	281	280	-1	20546	20337	1.0%
fib_rec	74	72	-2	69062	68075	1.4%
loop_sum	65	64	-1	7038	6838	2.8%
matmul	351	350	-1	24546	24295	1.0%
strops	226	223	-3	69086	67346	2.5%
合計	997	989	-8	190278	186891	1.8%

静的にはわずか 8 命令 (0.8%) しか減っていない。しかし動的には 1.8% 減っている。

いちばん効くのは `loop_sum` の 2.8% である。ループ本体が短いので、1 命令の差が相対的に大きい。逆に `matmul` は本体が 103 命令もあるので、1 命令減っても 1.0% にしかない。

`fib_rec` はループを 1 つも持たない (再帰だけ)。それでも 1.4% 減っているのは、回転ではなく「次の行へのジャンプ」の削除が効いているからである。O2 で「`fib_rec` の後方辺は 0」と分かっていたので、この結果は予言できていた。

11 発展課題

1. 入口の条件判定を省く: 多くのループは「最低 1 回はまわる」と分かっている (`for (i = 0; i < 200; ...)` など)。分かるなら入口の `j` すら要らない。どういう条件なら省けるか
2. ループ展開 (unrolling): 本体を 2 回ぶん並べると、条件判定の回数が半分になる。`loop_sum` で試して測る。コードサイズはどれだけ増えるか
3. .L3 の削除: `continue` が使われていないループでは、更新のラベルは要らない。ラベルが減るとブロックが減り、後の解析が速くなる
4. 分岐予測を考える: 現代の CPU は後方分岐を「成立する」と予測することが多い。回転後の `bnez` (後方へ戻る) は予測が当たりやすい。命令数以外の利得がここにある
5. B1 のピープホールと組み合わせる: `--passes isel,layout` と B1 のピープホールを両方かけるとどうなるか。順番で結果は変わるか

補足

コラム: 速さは命令数だけでは決まらない。 この回の最適化は、静的命令数をまったく減らさずに実行を速くした。逆のこともある — ループ展開はコードサイズを増やすが実行は速くなるし、命令数を減らしてもキャッシュミスが増えれば遅くなる。

だから最適化の効果は、**目的に合った物差しで測る**しかない。組み込み機器ならコードサイズ (静的)、計算機ならまず動的命令数、本当に知りたければ実測時間。O1 で 2 つの物差しを作ったのは、このためである。